

RETRIAL: Robust Encrypted Malicious Traffic Detection via Discriminative Relation Incorporation and Misleading Relation Correction

Jianjin Zhao[✉], *Member, IEEE*, Qi Li[✉], Zewei Han[✉], Junsong Fu[✉], Guoshun Nan[✉], *Member, IEEE*, Meng Shen[✉], *Member, IEEE*, and Bharat K. Bhargava[✉], *Life Fellow, IEEE*

Abstract—Encryption techniques greatly ensure the confidentiality and integrity of network communications. However, they also allow attackers to conceal malicious activities within encrypted traffic, posing severe cybersecurity challenges. Current detection methods primarily rely on statistics and correlation analysis. However, both statistical features and inter-entity relations can be easily obfuscated. Moreover, issues with low-quality data and fixed feature sets limit the generalizability and adaptability to defend against various evasion techniques. Robustifying encrypted malicious traffic detection in adverse conditions is still an open problem. In this paper, we propose RETRIAL, a robust encrypted malicious traffic detection system via discriminative relation incorporation and misleading relation correction. The key motivations behind RETRIAL are to accurately leverage the rich relations among flows for contextual analysis, and correct misleading ones for robust threat detection. Specifically, we construct a relational multigraph and develop a tailored Graph Attention Network (GAT) to selectively incorporate contextual information. Then we retrieve multi-order neighborhood similarity graphs as observations for adaptive relation correction. Following an iterative scheme, both detector performance and graph topology mutually optimize. To validate the robustness of RETRIAL, we simulate various adverse conditions by randomly dropping packets and greedily injecting perturbation edges. The experimental results show that RETRIAL is competitive in ideal condition. Under adverse conditions, though the performances of other state-of-the-art methods degrade significantly, RETRIAL consistently exhibits superior performance with a maximum reduction of only 5.88% in F1, highlighting its robustness in threat detection.

Index Terms—Encrypted traffic analysis, malicious traffic detection, graph representation learning.

Received 11 April 2024; revised 19 September 2024; accepted 15 October 2024. Date of publication 11 December 2024; date of current version 7 January 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62172055, Grant 61972039, Grant 62222201, Grant U20B2045, and Grant U23A20304; in part by the National Defense Basic Scientific Research Program of China under Grant JCKY2021602B002; in part by Beijing Nova Program under Grant 20220484174; and in part by Beijing Natural Science Foundation under Grant M23020. The associate editor coordinating the review of this article and approving it for publication was Dr. Daisuke Mashima. (*Corresponding author: Qi Li.*)

Jianjin Zhao, Qi Li, Zewei Han, Junsong Fu, and Guoshun Nan are with the Department of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: jianjinzhao@bupt.edu.cn; liqi2001@bupt.edu.cn; hanzewei@bupt.edu.cn; fujs@bupt.edu.cn; nanguo2021@bupt.edu.cn).

Meng Shen is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: shenmeng@bit.edu.cn).

Bharat K. Bhargava is with the Department of Computer Science, Purdue University, West Lafayette, IN 47906 USA (e-mail: bbshail@purdue.edu).

Digital Object Identifier 10.1109/TIFS.2024.3515821

1556-6021 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

I. INTRODUCTION

THE widespread adoption of encryption techniques in modern network applications significantly enhances the confidentiality and integrity of network communications. Nonetheless, it also retains a gray area for threat actors to conceal malicious activities within encrypted traffic, such as malware delivery, Command-and-Control (C&C) channels, and data exfiltration. The ubiquitous nature and inherent opacity of encrypted traffic pose a formidable challenge to network security management [1], [2], [3].

Many efforts have been made to detect encrypted malicious traffic [4], [5], [6]. Traditional methods based on decryption or signatures [7], [8], to varying degrees, present issues related to privacy violations and limited adaptability. In response, an increasing number of studies embrace machine learning as a viable solution [9], [10], [11]. These studies typically rely on statistical analysis of unencrypted fields and flow metadata to distinguish between benign and malicious traffic. However, threat activities such as Distributed Denial-of-Service (DDoS) attacks and ransomware campaigns may manifest in complex and distributed forms, rendering statistical analysis of individual flows insufficient.

Recently, another emerging trend involves in-depth correlation analysis across flows to reduce false positives and improve robustness. The interactions between network entities essentially constitute a graph structure, which has been explored in recent studies [12], [13], [14]. ST-Graph [4] captures spatio-temporal behaviors by graph representation learning. 3D-IDS [5] introduces a multi-layer graph diffusion method for dynamic intrusion detection. Additionally, RAPIER [15] and HYPERVISION [16] analyze various flows to mitigate label noise and extract robust interaction patterns respectively, achieving considerable progress.

In a nutshell, existing studies achieve accurate and robust detection primarily by improving training data quality and designing more resilient features, however, which are insufficient to tackle the continuously evolving threat landscape. First, the issue of low-quality testing data persists, undermining the model generalizability. Second, fixed feature sets limits the adaptability to cope with dynamic evasion strategies. Thus, we advocate to adaptively integrate and correct various interactions, mitigating the adverse effects induced by evasion techniques and generating more representative flow features for robust detection.

To fully exploit the potential of complex relations, three challenges need to be addressed: relation complexity, significance and reliability. Firstly, network entities may share multiple and various relations, while current solutions often-times oversimplify the complex interactions, failing to map their multifaceted nature. Secondly, the significance of different relations varies, requiring a systematic measurement to discriminatively incorporate them for fine-grained contextual information aggregation. Lastly, due to the prevalence of evasion techniques [17], [18], [19], [20], it is difficult to assess the reliability of relations. Potential spurious relations will inevitably lead to erroneous decision-making in detection.

In this paper, we develop a robust system, namely RETRIAL, that aims to detect encrypted malicious traffic via selectively exploring the rich relations among flows and correct the misleading of them. The key motivations behind RETRIAL are to accurately model the rich relations among flows, selectively leverage them for correlations, and correct those misleading ones to defend against obfuscation techniques for adaptive relation correction and robust threat detection.

Specifically, we first construct a relational multigraph incorporating semantic identities and behavioral similarities and record relation information as edge types to comprehensively model the complex interactions between flows. Then we develop a tailored Graph Attention Network (GAT) [21] to encode relation information into attention calculation and selectively incorporate contextual information. To correct those misleading relations, we retrieve multi-order neighborhood similarity graphs as observations, estimate and approximate optimal graph based on Bayesian inference and Expectation-Maximization (EM) algorithm. Following an iterative optimization scheme, RETRIAL realizes mutual optimization of both GNN encoder and graph topology. To evaluate the robustness of our system, we randomly drop packets and generate perturbation edges to simulate network fluctuations and adversarial attacks. Extensive experimental results show the superiority of our work under various adverse conditions.

In sum, the contributions of this paper are as follows:

- We propose RETRIAL, a practical encrypted malicious traffic detection system for robust threat detection and adaptive relation correction.
- We design a relational multigraph, and develop a tailored GAT to encode relation information into neighborhood aggregation for accurate contextual flow encoding.
- We correct misleading relations through Bayesian inference and EM algorithm, achieving mutual optimization of detector performance and graph topology iteratively.
- We simulate various adverse conditions by packet manipulation and edge perturbation. Extensive experimental results validate the superior robustness of RETRIAL.

II. RELATED WORK

In this section, we review the relevant studies on malicious traffic detection, which can be mainly divided into four categories, decryption-based, signature-based, statistics-based and correlation-based methods.

A. Decryption-Based Methods

Decryption-based methods decrypt encrypted contents for inspection, which reduce connection security, introduce additional attack vectors, and require additional computation time and delay. The common practice employed by enterprises is to deploy a MitM proxy between client and server. Durumeric et al. [22] investigate the potential vulnerabilities introduced by popular middle-boxes and security software, and conclude recommendations for both vendors and the security community. Instead of prematurely intercepting TLS connections, Wilkens et al. [23] advocate for a passive and transparent TLS decryption solution by selectively forward TLS key to trusted servers to enable selective decryption for forensic analysis. In any case, the idea of decryption-based methods stands in opposition to the fundamental principle of encryption techniques, raising issues concerning security and privacy.

B. Signature-Based Methods

Signature-based methods involve extracting signatures for known malicious traffic. Besides IP and domain block lists, Salesforce [7], [8] develops fingerprints tailored for TLS communications, including JA3, JA3S and JARM. JA3 and JA3S are passive observations for TLS clients and servers, while JARM is to actively scan to build a fingerprint for server-side applications. Despite their utility, the frequent updates and diverse sources of block lists complicate the fingerprinting process. To generate practical block lists, Ramanathan et al. [24] propose BLAG to aggregate multiple block lists feeds and produce an accurate and timely block list for detection. Dong et al. [25] propose MBTREE, a host-level network behavior fingerprint to detect encryption Trojans. Though signature-based detection methods are efficient and accurate, they cannot adapt to dynamic and unknown attack patterns.

C. Statistics-Based Methods

Statistics-based methods extract flow metadata as features and perform statistical analysis to fit a decision boundary between malicious and benign. Early studies primarily focus on TLS traffic. Anderson and McGrew [9], [10], [11] perform a series of measurement studies on observable features within TLS handshake message, highlighting its potential in malware analysis. They also emphasize the significance of statistical flow metadata for malicious traffic detection. On the basis, Wang et al. [26] leverage Convolutional Neural Network (CNN) to achieve malware traffic classification with the first 784 bytes of each flow. Fu et al. [27], [28] develop WHISPER by applying discrete Fourier transformation on per-packet feature sequences to extract sequential information in the frequency domain. As encryption protocols continue to advance, observable information diminishes. A notable example is the Encrypted Server Name Indication (ESNI) extension [29], which encrypts the server name requested by the client and keeps user browsing data private. Moreover, evasion techniques (e.g., domain fronting and fast-flux) easily obfuscate those statistical features, posing new challenges to detection.

TABLE I
COMPARISON WITH OTHER CORRELATION-BASED METHODS

Categories	Ref.	Various Relations	Selective Encoding	Adaptive Optimization	Robustness Implementation
n -tuple Aggregation	[30], [31]	×	×	×	×
Graph Modeling	[13]	×	×	×	×
	[32], [33]	✓	×	×	×
Robust Solution	[15]	×	✓	×	Label Correction
	[16]	✓	✓	×	Robust Features
	RETRIAL	✓	✓	✓	Interaction Correction

D. Correlation-Based Methods

Correlation-based methods improve detection capabilities by leveraging the rich relations among network entities. There are mainly two avenues for correlation analysis. One involves aggregating flow-level features, while the other constructs graphs to model entity relations. For example, Dai et al. [30] and Cui et al. [31] employ the former by aggregating 5-tuple flows to characterize network behaviors at service level (4-tuple) and channel level (2-tuple). For more flexible correlations, Zhao et al. [32], Li et al. [13] and Hong et al. [33] integrate various relations into an informative graph and utilize GNNs for analysis.

Recent studies have increasingly leveraged correlation analysis for robust detection. Qing et al. [15] estimate the distribution of training data to mitigate label noise in data preparation. Fu et al. [16] design robust graph structural features within a flow interaction graph in feature engineering. In contrast, our work focuses on adaptively correcting misleading interactions induced by various evasion techniques during both training and detection phases.

To achieve this, we construct a relational multigraph and record the relation type as the edge type to comprehensively capture the rich relations. On this basis, we employ a tailored GAT to selectively aggregate neighbor information considering relation types for contextual flow encoding. Moreover, we modify the graph topology to approximate the optimal graph estimated by Bayesian inference. Following an iterative optimization scheme, RETRIAL achieves adaptive relation correction and robust threat detection. Table I compares RETRIAL with other correlation-based methods.

III. PROBLEM DESCRIPTION

In this section, we first describe our threat model to determine the adversary's goals, capabilities and knowledge, and correspondingly set the design goal of RETRIAL. Then, we review common evasion techniques and specify those that can be employed by passive and proactive adversaries. With this background, we formally present the problem definition of encrypted malicious traffic detection under various adverse conditions.

A. Threat Model

We delineate between two types of adversaries: passive and proactive, based on the adversary's goals. Accordingly, their capabilities and knowledge exhibit notable distinctions.

1) *Passive Adversary*: Passive adversary's goal is to disrupt web service availability and overwhelm network bandwidth. Though passive adversary does not proactively seek to evade detection, the interference with network traffic leads to loss or distortion of critical features like packet lengths, thereby compromising the efficacy of detectors. In terms of capabilities, we assume that passive adversary can merely cause packet loss, and has no access to traffic data or detectors. Consequently, passive adversary performs black-box attacks, requiring no knowledge about the model structure or model feedback.

2) *Proactive Adversary*: Proactive adversary's goal is to generate adversarial traffic and exploit the vulnerabilities inherent in learning-based models to evade detection. Proactive adversary is able to utilize various evasion techniques to obfuscate the relations and mislead the decision of detectors. In this work, we propose to model the rich relations among flows as a graph and leverage GNN for detection. We assume that proactive adversary, armed with full knowledge about the graph and the model, performs white-box attacks. Notably, this assumption is impractical in real-world scenarios. We augment the proactive adversary's capabilities and knowledge to further validate the robustness of our solution.

To realize accurate and robust detection and tackle various adversaries, the design goals of RETRIAL are threefold.

a) *Elaborate Relation Incorporation*: It should be able to elaborately model the complex relations among flows for comprehensive correlation analysis.

b) *Discriminative Relation Encoding*: It should be able to discriminatively encode relation information to focus more on those relations that really matter in threat detection.

c) *Adaptive Relation Correction*: It should be able to adaptively correct misleading relations introduced by adversaries to defend against evasion techniques and realize robust detection.

B. Evasion Technique

To extend the life of threat campaigns, threat actors have developed many techniques to evade detection. Traditional encrypted malicious traffic detection methods primarily rely on unencrypted data fields (e.g., TLS handshake parameters) and statistical flow metadata (e.g., packet lengths, numbers and times). Recent studies further take the rich interactions among network entities (e.g., components and hosts) into consideration.

Here we briefly review several common evasion techniques that distort available features and correlations and refine the adversaries' capabilities, motivating us to cautiously analyze the rich relations within flows.

1) *DDoS Attack*: Distributed Denial-of-Service (DDoS) attacks coordinate multiple compromised hosts to overwhelm the target with massive volumes of traffic, rendering its service inaccessible to legitimate users. The numerous requests exhaust the targets' CPU and memory resources, resulting in significant performance degradation and packet loss.

2) *Packet Obfuscation*: Packet obfuscation techniques manipulate packets to distort statistical flow metadata. For example, Obfs4 bridge relay, a common pluggable transport in Tor, is able to pad packets with random sizes and split

TABLE II
THE EVASION TECHNIQUES EMPLOYED BY ADVERSARIES

Evasion Technique	Distorted Information	Adversary Type
DDoS Attack	Packet length, number and time	Passive
Packet Obfuscation	Packet length, number and time	Proactive
Fast Flux	IP address	Proactive
Distributed C&C	IP address and certificate	Proactive
Domain Fronting	IP address and certificate	Proactive

large packets into smaller packets. Additionally, recent studies [34], [35], [36] seek to adaptively modify packet features to obfuscate traffic analysis with minimal performance overhead.

3) *Fast Flux*: Fast flux techniques used by threat actors involve rapid shifting between IP addresses to delay or evade detection. In a fast flux network, multiple IP addresses are associated with a single domain name with frequent switches by altering Domain Name System (DNS) resource records, rendering IP-based blocking efforts difficult.

4) *Distributed C&C*: C&C architectures have evolved significantly, transitioning from centralized to Peer-to-Peer (P2P) and random. In centralized C&C, all compromised hosts are controlled by a single server following a client-server scheme. Conversely, P2P C&C empowers each infected host to work as a bot leader, distributing commands to other nodes. Taking one node down does not affect the whole botnet. Random C&C leverages various sources, such as social media, Content Delivery Network (CDN), and Internet Relay Chat (IRC), to issue commands, with better resilience against disruptions of key botnet components.

5) *Domain Fronting*: As a geographically distributed network that deploys edge nodes in different regions worldwide, CDN caches web resources in proxy servers and delivers content to users from the nearest nodes for faster responses. However, threat actors exploit the redirection capabilities of CDNs as domain fronts to obfuscate the intended destination of encrypted traffic (i.e., the IP address and the domain name of malicious services). Domain fronting has been actively abused to provide malicious content.

The evasion techniques employed by passive and proactive adversaries, along with their impact on information distortion, are summarized in Table II. Typically, passive adversary may conduct DDoS attacks, causing random packet loss, while proactive adversary may intentionally disguise the malicious behavioral patterns and manipulate correlations through various evasion techniques. These techniques illustrate the adversaries' potential capabilities, such as modifying per-packet features, switching IP addresses, and using legitimate server configurations like certificate. Although the adversaries may employ other techniques, the listed are sufficient to obfuscate comprehensive information in encrypted traffic analysis.

C. Problem Definition

In this paper, we aim to develop a robust encrypted malicious traffic detection system (i.e., REtrial) to defend against adversarial attacks and evasion techniques. The notations used in this paper are summarized in Table III.

Specifically, we construct a relational multigraph $\mathbf{G} = \{\mathbf{V}, \mathbf{E}, \mathbf{X}, \mathbf{R}\}$ with adjacency matrix set $\mathbf{A}$ to model the rich

TABLE III
NOTATIONS IN THIS PAPER

Notation	Description
$\mathbf{G}, \mathbf{A}$	The original multigraph and adjacency matrix set
$\mathbf{G}, \mathbf{S}$	The optimal multigraph and adjacency matrix set
$\mathbf{V}, \mathbf{E}, \mathbf{R}$	The node, edge and relation sets
$\mathbf{X}, \mathbf{Y}, \mathbf{Z}$	The node features, true and predicted labels
$\mathbf{W}, \mathbf{W}_r, \mathbf{W}_{res}$	The weight matrices for nodes, edges and residual connection
$\mathcal{N}_i$	The node set of node i 's neighbors
$\mathcal{E}_{ij}$	The edge set between node i and node j
$\mathbf{h}, \mathbf{e}$	The node features and edge types
e, r	An edge and a relation in $\mathbf{G}$
$\psi(\cdot)$	The edge type mapping function
$r_{\psi(e)}$	The learnable edge-type embedding for edge e
β_{ije}	The unnormalized attention scores of edge e
$\hat{\alpha}_{ije}$	β_{ije} with Softmax normalization
$\hat{\alpha}_{ij}$	The attention scores between node i and node j
α_{ij}	$\hat{\alpha}_{ij}$ with residual connection
γ	The scaling factor for residual connection
$\mathcal{H}, \mathcal{O}$	The intermediate node embedding and observation set
$\mathcal{Q}$	The estimated adjacency matrix set before sparsification
Ω	The stochastic block model parameters
α, β	The Bernoulli distribution parameters
c_i	The community of node i equal to true y_i or predicted z_i
M	The cardinality of the observation set $\mathcal{O}$
E_{ij}	The number of observations with edges between node i and node j
$q(\mathcal{G}^r)$	The probability distribution of the optimal graph $\mathcal{G}^r$

relations among flows, where $\mathbf{V}$ denotes the node set, $\mathbf{E}$ denotes the edge set, $\mathbf{X}$ denotes the node features, $\mathbf{R}$ denotes the edge type set, and $\mathbf{A} = \{\mathbf{A}' | r \in \mathbf{R}\}$ denotes the adjacency matrix set segregated by relations. It is notable that $\mathbf{G}$ is unreliable since there may exist spurious relations introduced by various obfuscation techniques.

Given the original relational multigraph $\mathbf{G}$ with adjacency matrix set $\mathbf{A}$ and partial labels $\mathbf{Y}$, the goal of REtrial is to train a GNN encoder $f_{\Theta}(\cdot)$ with parameter Θ for robust threat detection and estimate the optimal multigraph $\mathbf{G}$ with adjacency matrix set $\mathbf{S}$ to correct misleading relations. The objective function of REtrial can be formally defined as

$$\min_{\Theta, \mathbf{S}} \mathcal{L}(\mathbf{A}, \mathbf{X}, \mathbf{Y}) = \sum_{y_i \in \mathbf{Y}} \mathcal{L}(f_{\Theta}(\mathbf{S}, \mathbf{X})_i, y_i), \quad (1)$$

where $f_{\Theta}(\mathbf{X}, \mathbf{S})_i$ is the predicted label of node v_i given by the GNN encoder, and $\mathcal{L}(\cdot)$ is the loss function that measures the difference between the predicted and true labels.

IV. DESIGN DETAILS OF REtrial

A. Framework Overview

In this work, we propose REtrial to achieve robust encrypted malicious traffic detection in adverse conditions. As illustrated in Fig. 1, scReTrial consists of four components: relational multigraph construction, contextual flow encoding, Bayesian graph estimation, and iterative model optimization.

1) *Relational Multigraph Construction*: We construct a multigraph by modeling encrypted flows as nodes and rich relations as edges respectively. In the relational multigraph, the information available in encrypted traffic including unencrypted data fields and observable flow metadata is utilized to correlate flows. We establish connections between flows with identical semantic fields or similar communication patterns. The corresponding relation information is recorded as edge types for subsequent analysis. We will detail how to construct the relational multigraph in Section IV-B.

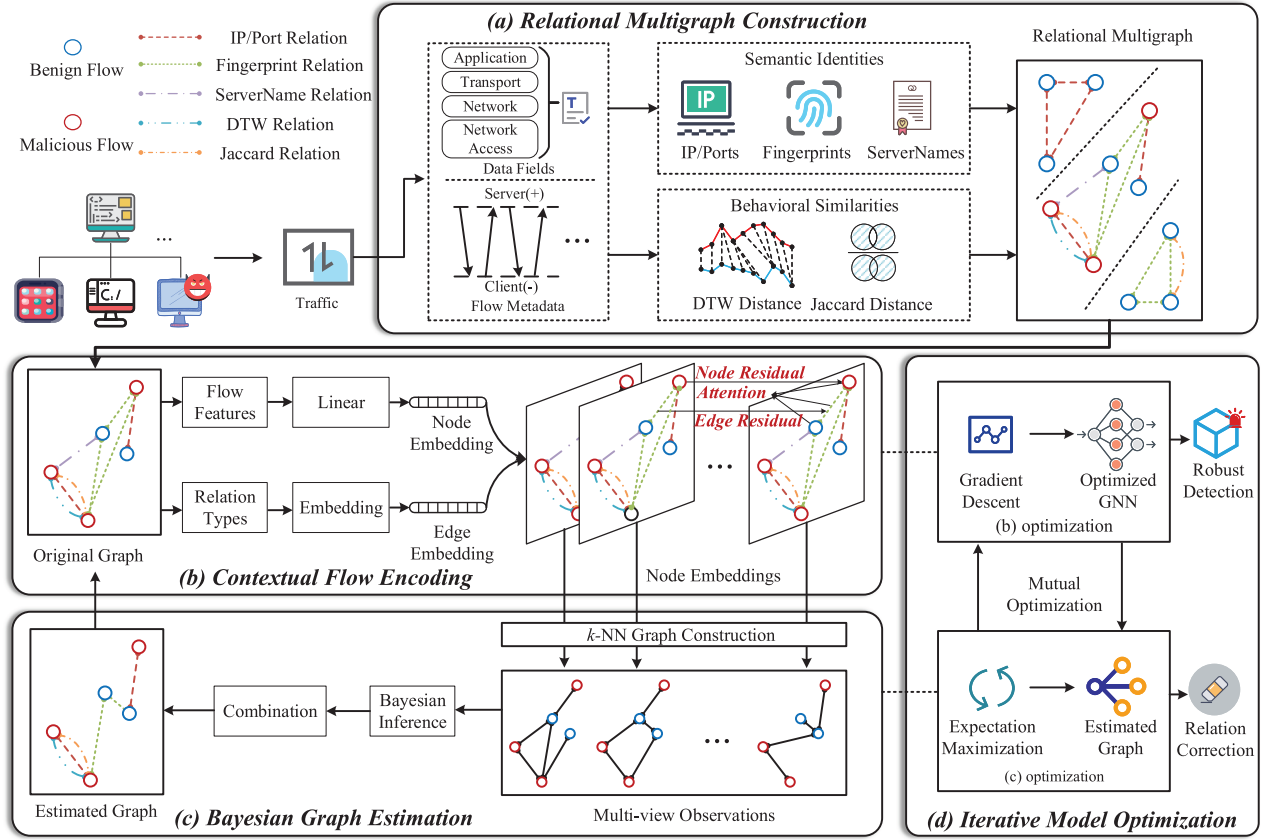


Fig. 1. The overview of our robust encrypted malicious traffic detection system RETRIAL.

2) *Contextual Flow Encoding:* To alleviate the adverse effects induced by evasion techniques, we selectively incorporate the neighborhood information of flows for contextual flow encoding. In particular, we adopt tailored Graph Attention Network (GAT) as an encoder. It extends original graph attention mechanism by considering edge types in importance calculation, enabling the assignment of importances to different nodes from various relations. We will present the tailored GAT layer for contextual flow encoding in Section IV-C.

3) *Bayesian Graph Estimation:* Assuming an optimal graph with relation corrected, it enables our encoder to derive better flow representations. We aim to estimate the optimal graph and modify our relational multigraph to approximate it. To this end, we compute multi-order neighborhood similarities to construct k -Nearest Neighbor (k -NN) graphs as observations to infer the optimal graph. Although they may contain errors when viewed individually, they can be combined to reflect the optimal graph. We calculate the probability of mapping the optimal graph to observations and utilize Bayesian inference to reverse it and estimate the posterior distribution. We will provide a detailed estimation process in Section IV-D.

4) *Iterative Model Optimization:* We follow an iterative optimization scheme to mutually optimize the relational multigraph and the GNN encoder. In each iteration, we first update the parameters of GNN encoder by typical gradient descent to obtain better observations. We then utilize Bayesian inference to derive the posterior probability of the optimal graph and maximize it with EM algorithm [37]. The iterative

optimization not only corrects those false relations but also promotes the detector performance. We will describe the details of iterative model optimization in Section IV-E.

B. Relational Multigraph Construction

To exploit the rich relations among flows for accurate threat detection, we consider the information available in encrypted traffic to construct a comprehensive relational multigraph $\mathbf{G}$, where flows are modeled as nodes and their rich relations are modeled as edges respectively.

1) *Node Feature Extraction:* In the relational multigraph, each node represents a bi-directional network flow. We construct a feature vector for each node/flow by considering both unencrypted data fields and statistical flow metadata.

a) *Unencrypted Data Fields:* Unencrypted Data Fields include critical information within the network and transport layers (e.g., IP address, port, transmission protocol, and TCP flags). Notably, TLS handshake messages in plaintext like ClientHello, ServerHello and Certificate enable client and server to authenticate each other and negotiate important parameters for subsequent encrypted communications, which relatively reflect the reliability of flows.

b) *Statistical Flow Metadata:* Statistical Flow Metadata is mainly derived from packet sequence information. We compute various statistical variants (e.g., mean, minimum, maximum, variance, standard deviation) of packet lengths and inter-arrival times to form statistical flow metadata. Though statistical flow metadata cannot provide deep insight about the

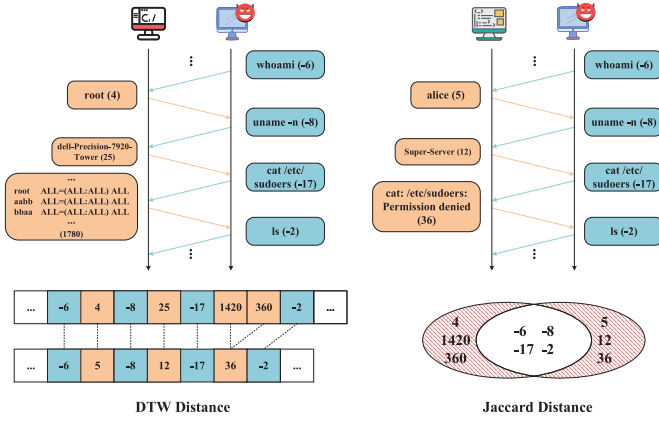


Fig. 2. A motivation example for computing DTW/Jaccard distance between packet length sequences. The responses from compromised hosts to identical commands vary due to different host configurations and environments. DTW distance captures the temporal similarities in sequences, accounting for the dynamic nature of packet transmission over time. Jaccard distance measures the overlap of unique packet lengths irrespective of their temporal order.

content of flows, it does facilitate inferences about the network behaviors of flows.

2) *Graph Topology Construction*: We consider the rich relations among flows to construct the graph topology. Referring to the node features, connections are established between flows that share the same semantic fields or similar communication patterns. There are in total five types of relations.

a) *IP/Port Relation*: We establish connections between flows with the same destination IP and destination port. The combination of IP address and port number identifies specific internet services and applications.

b) *Fingerprint Relation*: We establish connections between flows with the same JA3 and JA3S fingerprints. JA3 and JA3S are the MD5 hashes of the gathered fields in ClientHello and ServerHello messages. Salesforce has demonstrated that it is feasible to identify malicious communications by JA3&JA3S fingerprints.

c) *ServerName Relation*: We establish connections between flows with the same hostname in Server Name Indication (SNI) extension, which allows a server to host multiple services with different certificates on the same IP address and port number.

d) *Dynamic Time Warping (DTW) Relation*: We construct a k -NN graph for flows utilizing bidirectional packet length sequences, with DTW distance as the distance metric. DTW distance quantifies the similarity between two temporal sequences that may vary in speed.

e) *Jaccard Relation*: Similar to DTW relation, Jaccard distance gauges dissimilarity between sample sets by computing the difference between the sizes of the union and the intersection of two sets, divided by the size of the union.

We present an example in Fig. 2 to illustrate the motivation for using DTW and Jaccard distances as metrics to construct k -NN graphs, which complement each other, providing a comprehensive characterization of behavioral similarities.

The aforementioned simple graphs are merged into a comprehensive relational graph. Since flows may share more than one relations, our relational graph is a multigraph, allowing

for multiple edges (a.k.a. parallel edges) between nodes. We record relation information as edge types to construct relational multigraph for subsequent analysis.

C. Contextual Flow Encoding

In our relational multigraph, the complex relations among flows provide valuable contextual information for multi-flow correlation, however, which also bring noise to threat detection due to various evasion techniques. To selectively encode the contextual information into flow representations, we utilize a tailored graph attention network as an encoder. It extends original graph attention mechanism by considering both node features and edge types in importance calculation [38], enabling us to focus on those contextual flows that really matter.

Specifically, the encoder stacks tailored GAT layers to aggregate multi-order neighbor information and derive contextual flow representations for detection. In each layer, we maintain a learnable edge-type embedding $r_{\psi(e)}$ for each edge type $\psi(e) \in \mathbf{R}$, where $\psi(\cdot)$ is an edge type mapping function. The tailored GAT layer takes both node features $\mathbf{h}$ and edge types $\mathbf{e}$ as input and maps them into higher-level embeddings by linear transformations, respectively parameterized by weight matrix $\mathbf{W}$ and $\mathbf{W}_r$. Then we perform self-attention operation $a(\cdot)$ to compute attention scores by considering both node embeddings and edge type embeddings as

$$\hat{\beta}_{ije} = a(\mathbf{W}\mathbf{h}_i, \mathbf{W}\mathbf{h}_j, \mathbf{W}_r r_{\psi(e)}) \quad (2)$$

that indicate the importance of node j to node i when considering the type of edge e between them. To capture structural information, we compute $\hat{\beta}_{ije}$ for all nodes $j \in \mathcal{N}_i$ and all edges $e \in \mathcal{E}_{ij}$, where $\mathcal{N}_i$ is the neighbors of node i , and $\mathcal{E}_{ij}$ is the set of edges between node i and node j . Then we normalize the scores across all possible neighbors of node i using softmax function as

$$\hat{\alpha}_{ije} = \text{softmax}_j(\beta_{ije}) = \frac{\exp(\hat{\beta}_{ije})}{\sum_{k \in \mathcal{N}_i} \sum_{e \in \mathcal{E}_{ik}} \exp(\hat{\beta}_{ike})}. \quad (3)$$

We sum up the scores of all edges between node i and node j to measure the importance of node j to node i when considering all the relations as

$$\hat{\alpha}_{ij} = \sum_{e \in \mathcal{E}_{ij}} \hat{\alpha}_{ije}. \quad (4)$$

Following the original GAT, we implement self-attention operation a with a single-layer feed-forward neural network, parameterized by a weight vector $\mathbf{a}$, and LeakyReLU activation function. The normalized attention scores are fully expanded as

$$\hat{\alpha}_{ij} = \frac{\sum_{e \in \mathcal{E}_{ij}} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j \parallel \mathbf{W}_r r_{\psi(e)}]))}{\sum_{k \in \mathcal{N}_i} \sum_{e \in \mathcal{E}_{ik}} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k \parallel \mathbf{W}_r r_{\psi(e)}]))}. \quad (5)$$

To derive better flow representations, we adopt several tricks to enhance the encoder's capability, including edge residual connection, node residual connection, and multi-head attention. Here we formally define these techniques.

1) *Edge Residual Connection*: We add residual connections to the raw attention scores $\hat{\alpha}$ across layers since residual attention scheme has been demonstrated effective in previous studies. The final attention scores at the l^{th} layer can be expressed as

$$\alpha_{ij}^{(l)} = (1 - \gamma) \hat{\alpha}_{ij}^{(l)} + \gamma \alpha_{ij}^{(l-1)}, \quad (6)$$

2) *Node Residual Connection*: We also add residual connection to the node embeddings across layers to mitigate the over-smoothing issue in GNN. The aggregation process (i.e., linear combination of the neighbors and ELU activation function) at the l^{th} layer can be expressed as

$$h_i^{(l)} = \text{ELU} \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^{(l)} \mathbf{W}^{(l)} h_j^{(l-1)} + \mathbf{W}_{res}^{(l)} h_i^{(l-1)} \right), \quad (7)$$

where $\mathbf{W}_{res}$ is a learnable weight matrix to unify the dimensions of node embeddings across layers.

3) *Multi-Head Attention*: We extend the self-attention operation to be multi-head by repeating the computations and aggregations K times independently. Correspondingly, Equation (6-7) are updated as

$$\alpha_{ijk}^{(l)} = (1 - \gamma) \hat{\alpha}_{ijk}^{(l)} + \gamma \alpha_{ijk}^{(l-1)}, \quad (8)$$

$$h_i^{(l)} = \text{ELU} \left(\sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ijk}^{(l)} \mathbf{W}_k^{(l)} h_j^{(l-1)} + \mathbf{W}_{res_k}^{(l)} h_i^{(l-1)} \right). \quad (9)$$

D. Bayesian Graph Estimation

Through fine-grained attention computation, the encoder focus more on important neighbors to derive flow representations, alleviating the adverse effects of misleading relations to a great extent. Assume that there exists an optimal graph with relations corrected, we aim to modify our relational multigraph to approximate it.

1) *Graph Generation*: Here is the question: *how to assume the generation of the optimal graph to correct those spurious edges and achieve better performance?* A naive idea may be removing those edges between malicious nodes and benign nodes, which requires the optimal graph to exhibit latent community structure. Stochastic Block Model (SBM) as a generative model for graphs, is a good choice to guide the underlying structure generation of the optimal graph, where edges are more common within communities than between communities. In other words, the probability of an edge's existence between two nodes depends only on their community attributions (i.e., true labels and predicted labels).

Formally, the optimal multigraph $\mathcal{G}$ can be segregated into a set of simple graphs $\{\mathcal{G}^r | r \in \mathbf{R}\}$ by relations. The generation of each optimal relational simple graph $\mathcal{G}^r$ takes the form of a probability distribution $P(\mathcal{G}^r | \Omega^r, \mathbf{Y}, \mathbf{Z})$, where Ω^r denotes SBM parameters, $\Omega_{c_i c_j}^r$ denotes the probability that an edge exists between node v_i within community c_i and node v_j within community c_j , $\mathbf{Z}$ denotes predicted labels, and $\mathbf{Y}$ denotes true labels. Given Ω^r , $\mathbf{Y}$, and $\mathbf{Z}$, the probability of generating the optimal simple graph $\mathcal{G}^r$ for relation r is formalized as

$$P(\mathcal{G}^r | \Omega^r, \mathbf{Y}, \mathbf{Z}) = \prod_{i < j} \Omega_{c_i c_j}^r \mathcal{G}_{ij}^r \left(1 - \Omega_{c_i c_j}^r \right)^{1 - \mathcal{G}_{ij}^r}, \quad (10)$$

where $\mathcal{G}_{ij}^r \in \{0, 1\}$ denotes whether the edge between node v_i and node v_j exists in $\mathcal{G}^r$, and c_i equals true label y_i if node v_i is in the training set, otherwise predicted label z_i .

In graph generation, we have assumed the underlying structure of the optimal graph as a prior constraint. Next, we introduce some external observations to infer the optimal graph.

2) *Observation Mapping*: Though the original graph topology and the multi-order neighborhood similarities during aggregations may be unreliable or conflicting, they reflect the optimal graph from different perspectives. We treat them as external observations, combine them to reduce biases, and establish connections between these observations and the optimal graph.

During contextual flow encoding, we aggregate rich contextual information through a GNN encoder, which stacks tailored GAT layers to aggregate multi-order neighbor information. The node embeddings from its hidden layers capture the structural information from local to global. We consider all the intermediate node embeddings to compute multi-order neighborhood similarities as multi-view observations, which are importance evidences for inferring the optimal graph.

We take out the intermediate node embeddings across layers $\mathcal{H} = \{\mathbf{H}^{(0)}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(l)}\}$ to construct k -NN graphs. Combined with the original graph $\mathbf{A}$, we form the multi-view observation set $\mathcal{O}$ as

$$\mathcal{O} = \{\mathbf{A}, \mathbf{O}^{(0)}, \mathbf{O}^{(1)}, \dots, \mathbf{O}^{(l)}\}, \quad (11)$$

where $\mathbf{O}^{(i)}$ denotes the k -NN graph generated by $\mathbf{H}^{(i)}$, characterizing i -order neighborhood similarity.

These observations provide multifaceted insights into the optimal graph, raising another question: *how to map the available observations onto the optimal graph?* It is challenging to accurately infer the optimal graph from these observations due to their potential unreliability and conflicts. However, conversely given the optimal graph, mapping it onto these observations is relatively straightforward, which has been proven feasible [39], [40].

Specifically, we assume that the edge observations are independent Bernoulli random variables conditioned on the ground truth $\mathcal{G}_{ij}^r$ in the optimal graph with a true positive rate α^r and a false positive rate β^r . Suppose that we observe E_{ij} edges between node v_i and v_j out of the M (i.e., $|\mathcal{O}|$) observations, the probability of mapping the optimal simple graph $\mathcal{G}^r$ for relation r to the observations is formalized as

$$P(\mathcal{O} | \mathcal{G}^r, \alpha^r, \beta^r) = \prod_{i < j} [\alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}}] \mathcal{G}_{ij}^r \times [\beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}}]^{1 - \mathcal{G}_{ij}^r}. \quad (12)$$

After successfully generating the optimal graph constrained by SBM and establishing connections between the observations and the optimal graph, we proceed to present our graph estimation process based on Bayesian inference.

3) *Graph Estimation*: To estimate the optimal graph $\mathcal{G}$, we collect the available information including true labels $\mathbf{Y}$, predicted labels $\mathbf{Z}$, and observations $\mathcal{O}$ as prior knowledge. Additionally, we constrain the underlying structure of the optimal graph by introducing SBM parameters Ω^r in graph

generation, and establish connections between the observations and the optimal graph by introducing the parameters of Bernoulli distribution α^r and β^r in observation mapping.

With these preliminary preparations, the posterior probability $P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})$ can be calculated with Equation (10)–(12) by applying Bayes' theorem as

$$P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O}) = \frac{P(\mathcal{O} | \mathcal{G}^r, \alpha^r, \beta^r) P(\mathcal{G}^r | \Omega^r, \mathbf{Y}, \mathbf{Z}) P(\Omega^r) P(\alpha^r) P(\beta^r)}{P(\mathbf{Y}, \mathbf{Z}, \mathcal{O})}, \quad (13)$$

where $P(\Omega^r)$, $P(\alpha^r)$, $P(\beta^r)$, and $P(\mathbf{Y}, \mathbf{Z}, \mathcal{O})$ are the probabilities of the parameters and those available information, which are assumed to be independent.

By summing up the probabilities of all possible structures of the optimal graph $\mathcal{G}^r$, we get the posterior probabilities of parameters Ω^r , α^r , and β^r as

$$P(\Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O}) = \sum_{\mathcal{G}^r} P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O}). \quad (14)$$

As such, we introduce latent variables $q(\mathcal{G}^r)$ to denote the probability of the optimal simple graph $\mathcal{G}^r$ for relation r . By finding maximum likelihood of parameters Ω^r , α^r , and β^r , we determine the distribution of latent variables $q(\mathcal{G}^r)$ and estimate the adjacency matrix $\mathcal{Q}^r$ for the optimal simple graph $\mathcal{G}^r$ for relation r as

$$\mathcal{Q}_{ij}^r = \sum_{\mathcal{G}^r} q(\mathcal{G}^r) \mathcal{G}_{ij}^r, \quad (15)$$

where $\mathcal{Q}_{ij}^r \in [0, 1]$ denotes the probability that an edge between node v_i and node v_j exists in the optimal simple graph $\mathcal{G}^r$ for relation r .

The aforementioned procedure is applied for each relation $r \in \mathbf{R}$ to estimate the adjacency matrix $\mathbf{S}^r$ of the optimal simple graph $\mathcal{G}^r$. Subsequently, we combine them to derive the optimal multigraph $\mathcal{G}$ to better depict the rich relations among flows.

E. Iterative Model Optimization

In this section, we introduce our iterative optimization scheme to alternatively optimize the GNN encoder and the relational multigraph. This mutual optimization workflow facilitates adaptive relation correction and robust threat detection.

1) *GNN Encoder Optimization*: We construct a relational multigraph to model the rich relations among flows, and transform encrypted malicious traffic detection problem into a node classification task. For semi-supervised node classification, we compute the cross-entropy loss over all labeled nodes in the training set as

$$\min_{\Theta} \mathcal{L}(\mathbf{A}, \mathbf{X}, \mathbf{Y}) = - \sum_{y_i \in \mathbf{Y}} y_i \ln z_i. \quad (16)$$

Guided by labeled nodes, we minimize the loss to optimize GNN parameters Θ through gradient descent, which not only yields better flow representations for threat detection, but also derives more accurate observations for graph optimization.

2) *Relational Multigraph Optimization*: To estimate the optimal multigraph $\mathcal{G}$, we perform an independent estimation on its subgraphs $\{\mathcal{G}^r | r \in \mathbf{R}\}$ segregated by relations. Specifically, we first introduce its probability distribution $q(\mathcal{G}^r)$ as latent variables. Subsequently, we employ EM algorithm to perform maximum likelihood estimation of the probability of Equation (14) in the presence of latent variables $q(\mathcal{G}^r)$. EM algorithm iteratively determines the probability distribution of latent variables $q(\mathcal{G}^r)$ and update the values of parameters Ω^r , α^r , and β^r , allowing for a comprehensive exploration of the underlying structure of the optimal graph. Following typical EM algorithm, relational multigraph optimization involves iterative application of *Expectation step (E-step)* and *Maximization step (M-step)*.

E-step. It is more convenient to maximize the log of the likelihood function. Because the logarithm is monotonically increasing function of its argument, maximization of the log of a function is equivalent to maximization of the function itself. Since $\log(\cdot)$ is a convex function, using Jensen's inequality for a random variable x , we have

$$\log \mathbb{E}(x) \geq \mathbb{E}(\log x). \quad (17)$$

Referring to Equation (17), we rewrite the complete log likelihood function of Equation (14) by multiplying it by $\frac{q(\mathcal{G}^r)}{q(\mathcal{G}^r)}$ to create a lower bound as

$$\begin{aligned} \log P(\Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O}) &= \log \sum_{\mathcal{G}^r} \frac{q(\mathcal{G}^r)}{q(\mathcal{G}^r)} P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O}) \\ &= \log \mathbb{E} \left(\frac{P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}{q(\mathcal{G}^r)} \right) \\ &\geq \mathbb{E} \left(\log \frac{P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}{q(\mathcal{G}^r)} \right) \\ &= \sum_{\mathcal{G}^r} q(\mathcal{G}^r) \log \frac{P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}{q(\mathcal{G}^r)}. \end{aligned} \quad (18)$$

To make the bound tight (i.e., make the inequality hold with equality), we require that the random variable as

$$\log \frac{P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}{q(\mathcal{G}^r)} = c, \quad (19)$$

where c is a constant that does not depend on $\mathcal{G}^r$. Since $q(\mathcal{G}^r)$ is the probability distribution of the optimal simple graph $\mathcal{G}^r$ for relation r that satisfies $\sum_{\mathcal{G}^r} q(\mathcal{G}^r) = 1$, we have

$$q(\mathcal{G}^r) = \frac{P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}{\sum_{\mathcal{G}^r} P(\mathcal{G}^r, \Omega^r, \alpha^r, \beta^r | \mathbf{Y}, \mathbf{Z}, \mathcal{O})}. \quad (20)$$

Substituting Equation (10) into Equation (20), we get the expression for $q(\mathcal{G}^r)$ as (21), as shown at the bottom of the next page.

In *E-step*, we create the lower bound of log-likelihood by applying Jensen's inequality. To hold equality, we make the random variable constant and determine the probability distribution $q(\mathcal{G}^r)$ of the optimal simple graph $\mathcal{G}^r$.

M-step. To push the lower bound and maximize the log likelihood, we take derivatives with respect to each parameter

Ω^r , α^r , and β^r for the right side of Equation (18) while holding $q(\mathcal{G}^r)$ constant as

$$\sum_{\mathcal{G}^r} q(\mathcal{G}^r) \sum_{i < j} \left[\frac{\mathcal{G}_{ij}^r}{\Omega_{c_i c_j}^r} - \frac{1 - \mathcal{G}_{ij}^r}{1 - \Omega_{c_i c_j}^r} \right] = 0, \quad (22)$$

$$\sum_{\mathcal{G}^r} q(\mathcal{G}^r) \sum_{i < j} \mathcal{G}_{ij}^r \left[\frac{E_{ij}}{\alpha^r} - \frac{M - E_{ij}}{1 - \alpha^r} \right] = 0, \quad (23)$$

$$\sum_{\mathcal{G}^r} q(\mathcal{G}^r) \sum_{i < j} (1 - \mathcal{G}_{ij}^r) \left[\frac{E_{ij}}{\beta^r} - \frac{M - E_{ij}}{1 - \beta^r} \right] = 0. \quad (24)$$

By swapping the order of summations, we eliminate $\mathcal{G}_{ij}^r$ to independently derive simple expressions of parameters Ω^r , α^r , and β^r as

$$\Omega_{xy}^r = \begin{cases} \frac{N_{xy}}{n_x n_y} & \text{if } x \neq y, \\ \frac{2N_{xx}}{n_x(n_x - 1)} & \text{otherwise,} \end{cases} \quad (25)$$

where

$$\begin{aligned} n_x &= \sum_i \mathbb{I}(c_i = x) \\ N_{xy} &= \sum_{i < j} \mathcal{Q}_{ij}^r \mathbb{I}(c_i = x) \mathbb{I}(c_j = y), \\ \alpha^r &= \frac{\sum_{i < j} \mathcal{Q}_{ij}^r E_{ij}}{M \sum_{i < j} \mathcal{Q}_{ij}^r}, \end{aligned} \quad (26)$$

$$\beta^r = \frac{\sum_{i < j} (1 - \mathcal{Q}_{ij}^r) E_{ij}}{M \sum_{i < j} (1 - \mathcal{Q}_{ij}^r)}, \quad (27)$$

In *M-step*, we find parameters Ω^r , α^r , and β^r to maximize the log likelihood by differentiating and solving for each parameters respectively.

With updated Ω^r , α^r , and β^r , we substitute Equation (21) into Equation (15) to estimate the adjacency matrix $\mathcal{Q}^r$ for the optimal simple graph $\mathcal{G}^r$ as

$$\mathcal{Q}_{ij}^r = \frac{\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}}}{\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}} + (1 - \Omega_{c_i c_j}^r) \beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}}}. \quad (28)$$

In this way, we estimate the adjacency matrix $\mathcal{Q}^r$ of $\mathcal{G}^r$ using Equation (28). The probability distribution $q(\mathcal{G}^r)$ of the optimal simple graph $\mathcal{G}^r$ can be rewritten as

$$q(\mathcal{G}^r) = \prod_{i < j} \mathcal{Q}_{ij}^r \mathcal{G}_{ij}^r (1 - \mathcal{Q}_{ij}^r)^{1 - \mathcal{G}_{ij}^r}. \quad (29)$$

We alternate between *E-step* and *M-step* until a convergence criteria (i.e., the difference between the updated and the previous parameters is less than tolerance λ) is met.

To improve computational efficiency and reduce noise, we eliminate edges with low confidence scores by masking out those elements smaller than a non-negative threshold ϵ to get a sparse adjacency matrix $\mathcal{S}^r$ as

$$\mathcal{S}_{ij}^r = \begin{cases} 1 & \mathcal{Q}_{ij}^r \geq \epsilon \\ 0 & \text{otherwise.} \end{cases} \quad (30)$$

Given the updated multigraph with homophily, the GNN encoder can be further optimized to exploit the benefits of the refined topology and ensure that the encoder is better aligned with the updated multigraph, potentially improving its performance on unseen data.

3) *Workflow Overview*: With a detailed exploration of the aforementioned components, we now present the complete workflow of RETRIAL to integrate these standalone components into a cohesive system. During the preparation phase, raw traffic captures are split into 5-tuple flows, and flow-level features, including data fields and flow metadata are extracted to construct the relational multigraph as the original input. The training process is outlined in Algorithm 1. Initially, all parameters of RETRIAL are randomly initialized for subsequent iterative optimization. In each iteration, we first update the GNN encoder parameters Θ to form observations $\mathcal{O}$ as evidences, and then independently estimate the adjacency matrices $\mathcal{S}$ of its subgraphs segregated by relations $\mathbf{R}$ using EM algorithm. Following this, a sparsification step is applied with a threshold ϵ to refine the relational multigraph. After τ iterations, this workflow yields a reliable relational multigraph with corrected relation along with a well-trained GNN encoder capable of robust detection.

V. EXPERIMENTS

A. Experimental Setup

1) *Datasets*: We validate RETRIAL's performance and robustness on three datasets: *CTU*, *MTA*, and *USTC-TFC2016* datasets. Table IV summarizes the details of these datasets.

CTU dataset, released by Czech Technical University, serves as a benchmark for encrypted malicious traffic detection tasks, which comprises both normal and malware traffic across various scenarios. This dataset consists of 130,716 flows with a total size of 21.6 GB.

MTA dataset is a collection of traffic captures related to malware infections. We reserve encrypted flows as positive

$$\begin{aligned} q(\mathcal{G}^r) &= \frac{\prod_{i < j} \left[\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}} \right]^{\mathcal{G}_{ij}^r} \left[(1 - \Omega_{c_i c_j}^r) \beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}} \right]^{1 - \mathcal{G}_{ij}^r}}{\sum_{\mathcal{G}^r} \prod_{i < j} \left[\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}} \right]^{\mathcal{G}_{ij}^r} \left[(1 - \Omega_{c_i c_j}^r) \beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}} \right]^{1 - \mathcal{G}_{ij}^r}} \\ &= \prod_{i < j} \frac{\left[\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}} \right]^{\mathcal{G}_{ij}^r} \left[(1 - \Omega_{c_i c_j}^r) \beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}} \right]^{1 - \mathcal{G}_{ij}^r}}{\Omega_{c_i c_j}^r \alpha^{r E_{ij}} (1 - \alpha^r)^{M - E_{ij}} + (1 - \Omega_{c_i c_j}^r) \beta^{r E_{ij}} (1 - \beta^r)^{M - E_{ij}}}. \end{aligned} \quad (21)$$

Algorithm 1 The Optimization Algorithm of RETRIAL

Input: original adjacency matrices $\mathbf{A}$, relations $\mathbf{R}$, and partial labels $\mathbf{Y}$.
Input: hyper-parameters: k in k -NN, iterations τ , tolerance λ , and threshold ϵ .
Output: optimal adjacency matrices $\mathcal{S}$, and GNN encoder parameters Θ .

```

1 Randomly initialize  $\Theta$ ,  $\Omega$ ,  $\alpha$ , and  $\beta$ ;
2  $\mathcal{Q} = \mathbf{A}$ ;
  // Iterative Optimization;
3 for  $i = 1$  to  $\tau$  do
  // GNN Encoder Optimization;
4   Update  $\Theta$  via Equation (16); // GNN Update;
5   Form  $\mathcal{O}$  via Equation (11);
  // Relational Multigraph Optimization;
6   for  $r$  in  $\mathbf{R}$  do
    // Expectation-Maximization Algorithm;
7     while  $|\alpha^r - \alpha_{old}^r| > \lambda$  or  $|\beta^r - \beta_{old}^r| > \lambda$  do
8        $\alpha_{old}^r = \alpha^r$ ;
9        $\beta_{old}^r = \beta^r$ ;
10      Update  $\Omega^r$ ,  $\alpha^r$ , and  $\beta^r$  via Equation (25-27);
      // M-Step;
11      Update  $\mathcal{Q}^r$  via Equation (28); // E-Step;
12    end
13  end
  // Sparsification;
14  Extract  $\mathcal{S}$  from  $\mathcal{Q}$  via Equation (30);
15   $\mathbf{A} = \mathcal{S}$ ; // Graph Update;
16 end
17 return  $\mathcal{S}$  and  $\Theta$ ;
```

TABLE IV
DATASET DESCRIPTIONS

Dataset	#Flow		#PCAP	#Class
	Benign	Malicious		
CTU	61,428	69,288	59	2
MTA	361,113	121,967	2989	2
USTC-TFC2016	148,365	239,672	26	20

and capture encrypted traffic of benign applications from a backbone router of the campus network as negative. It consists of 483,080 flows with a total size of 106 GB.

USTC-TFC2016 dataset is released by University of Science and Technology, China for malware traffic detection evaluation, which contains the network traces of ten types of benign applications and ten types of malware. It consists of 388,037 flows with a total size of 3.7 GB.

2) *Baselines*: To comprehensively evaluate the effectiveness of RETRIAL, we select seven baselines, including three single-flow analysis methods (i.e., JOY [10], USTC-TK [26], and CICFLOW [41]) and four multi-flow correlation methods (i.e., MALDISCOVERY [33], RS-GAT [32], RAPIER [15] and HYPERVISION [16]), where RAPIER and HYPERVISION are also robust detection solutions designed to mitigate label noise in low-quality training datasets and learn flow interac-

tion patterns to derive resilient representations respectively. In contrast, RETRIAL explicitly addresses interactions noise, providing a direct defense against advanced threats.

3) *Metrics*: Following common practice in previous studies [5], [16], we select F1-score (F1) and the area under the ROC curve (AUC) as evaluation metrics.

4) *Setup*: RETRIAL is deployed on a DELL server equipped with a 24-core Intel Xeon 6240R CPU @2.40GHz, Ubuntu 20.04, 64 GB memory, and 2×NVIDIA RTX A5000 GPUs. To obtain more reliable results, we repeat each experiment ten times to calculate the average and standard deviation of evaluation metrics.

We divide each of the three datasets into training and testing sets with a 7:3 ratio. To address the common pitfall of data snooping in learning-based security systems, we construct the relational multigraphs separately for training and testing data. The corresponding adjacency matrices $\mathbf{A}_{train}$ and $\mathbf{A}_{test}$ are concatenated into block diagonal matrices as

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{train} & 0 \\ 0 & \mathbf{A}_{test} \end{bmatrix}, \quad (31)$$

where the off-diagonal blocks (i.e., the lower-left and upper-right corners) contain only zeros. Instead of directly constructing adjacency matrices with both training and testing data, this data organization ensures that no information from testing data is exposed during the training phase, effectively preventing data snooping. During iterative optimization, potential connections between training and testing data may emerge within the updated graph topology. To mitigate this data snooping, we mask the upper-right corner block. Here, data snooping specifically refers to test snooping, and it should be noted that temporal and selective snooping pitfalls are not a concern in this work.

To validate the robustness of RETRIAL, we define passive and proactive adversaries and simulate two attack scenarios to map them. To simulate a passive adversary, we perform black-box attacks by randomly dropping packets with a packet loss rate $r_l \in \{10\%, 20\%, 30\%\}$. To simulate a proactive adversary, we conduct white-box attacks by greedily injecting perturbation edges into relational multigraph with an edge perturbation rate $r_p \in \{5\%, 10\%, 15\%\}$ by computing the gradient of the loss function with respect to the graph structure.

To assess the scalability of RETRIAL in handling general encrypted traffic, we exclusively parse raw bytes for *USTC-TFC2016* dataset and construct a relational multigraph without considering fingerprint and certificate relations.

5) *Implementation*: RETRIAL is primarily developed in C++ (GCC version 11.4.0) and Python (version 3.8.18). To promote transparency and repeatability, we make the source code of RETRIAL public.¹

For relational multigraph construction, RETRIAL extracts features with PcapPlusPlus (version 22.05) and computes DTW distances with dtaidistance (version 2.3.10). Then it leverages PyTorch (version 1.6.0) and DGL (version 0.6.1) to implement a tailored graph attention network to selectively encode contextual information. In Bayesian graph estimation,

¹<https://github.com/MetaRockETC/ReTrial>

TABLE V
PERFORMANCE COMPARISONS OF RETRIAL AND BASELINES IN IDEAL CONDITIONS

METHODS	<i>CTU</i> (binary)		<i>MTA</i> (binary)		<i>USTC-TFC2016</i> (multi-class)	
	F1	AUC	F1	AUC	F1	AUC
JOY	98.56±0.07	99.19±0.03	98.72±0.14	99.76±0.06	99.18±0.04	99.68±0.02
USTC-TK	98.94±0.01	99.92±0.02	99.55±0.13	99.32±0.16	99.42±0.02	99.83±0.01
CICFLOW	98.38±0.58	99.86±0.04	98.89±0.14	99.98±0.01	99.12±0.02	99.85±0.01
RS-GAT	97.33±0.04	98.47±0.09	98.63±0.16	99.45±0.10	98.15±0.26	98.82±0.04
MALDISCOVERY	98.76±0.05	99.33±0.05	98.42±0.07	99.66±0.09	98.25±0.03	99.88±0.01
RAPIER	99.06±0.61	99.20±0.43	99.59±0.37	99.48±0.31	-	-
HYPERVISION	98.29±0.00	98.61±0.00	98.49±0.00	99.63±0.00	-	-
RETRIAL	98.83±0.09	99.74±0.04	99.60±0.09	99.72±0.01	99.20±0.06	99.77±0.01

We highlight the best in ●, the second best in ●, and the third best in ●.

it derives k -NN graphs as evidence using Scikit-learn (version 1.3.2). Finally, we implement the EM solver and iteratively optimize the relational multigraph and the GNN encoder to achieve adaptive relation correction and robust threat detection.

B. Performance Analysis

In this subsection, we evaluate the performance of RETRIAL and other baselines on three datasets in terms of both efficacy and efficiency.

1) *Efficacy*: Table V summarizes the experimental results of all methods in ideal conditions. In general, RETRIAL and all baselines perform well on TLS and general encrypted traffic datasets in ideal conditions. Specifically, RAPIER, USTC-TK, and RETRIAL are basically the top three models with minimal performance differences.

A notable observation is that statistical analysis methods are highly competitive, which suggests that single-flow features including data fields, flow metadata, and raw bytes remain effective in ideal conditions without adverse influences. However, HYPERVISION has demonstrated that single-flow features can be easily obfuscated by typical evasion strategies. Moreover, the superiority of most multi-flow correlation methods, especially HYPERVISION is not as evident as expected. In some cases, incorporating relations even degrades performance. RS-GAT and MALDISCOVERY that model relations among flows as graphs, fail to achieve the expected performance improvements but results in a negative gain.

To find out why multi-flow correlation methods are unsatisfactory, we review the traffic data, extracted features, and correlations. The reasons are twofold. First, partial correlations are disturbances. For example, in *CTU* dataset, both RS-GAT and RETRIAL consider IP/Port relation. We count 3,891 unique destinations in the benign part and 7,035 unique destinations in the malicious part, with 357 overlapping between them. Similarly, for fingerprint and ServerName relations, we observe 87 and 554 overlaps, respectively. These overlaps introduce noisy interactions, and mislead decisions. Second, label noise significantly affects performance. The overlap in server names between benign and malicious includes legitimate websites like “accounts.google.com” and “en.wikipedia.org”, which explains why single-flow analysis methods perform better than most correlation analysis methods, since correlations amplify the noise. Also, it highlights why RAPIER outper-

forms HYPERVISION and RETRIAL, since RAPIER effectively correct those noisy labels. Although HYPERVISION’s graph topology and flow interactions basically remain unaffected by these issues, the noise affects the final evaluations.

We also note that though RETRIAL introduce more interactions including noise, it outperforms most baselines. The reason behind this is that RETRIAL selectively aggregates relation information and iteratively correct those misleading relations to mitigate noise. To further improve the performance of RETRIAL, two key strategies can be considered. First, more rigorous data cleaning is necessary. Instead of broadly labeling entire PCAP files as benign or malicious, finer-grained labels should be assigned based on flow-level features, particularly for flows associated with clearly legitimate or suspicious server names. This would minimize labeling errors and reduce noise. HYPERVISION excels in this regard with its fine-grained packet/flow-level labeling. Second, graph construction must be approached carefully. The use of reliable access lists or block lists to avoid noisy interactions between benign and malicious flows is essential for improving detection performance.

2) *Efficiency*: We evaluate the efficiency of RETRIAL with two state-of-the-art robust detection methods (i.e., RAPIER and HYPERVISION) across different phases: preparation, training, and detection. Following the experimental setup in RAPIER, we take a subset of *CTU* dataset and set the training size as 500. The time overheads are presented in Table VII.

In summary, HYPERVISION demonstrates faster performance than RAPIER and RETRIAL mainly for two reasons. First, HYPERVISION is developed in C++, which offers better computational efficiency than RAPIER and RETRIAL developed in Python. Second, their insights to achieve robustness are fundamentally different. HYPERVISION aims at modeling flow interaction patterns to learn graph structural features, whereas, RAPIER and RETRIAL focus on adaptive optimizations to mitigate label noise and interaction noise, respectively.

The preparation phase includes feature extraction, label correction and data augmentation for RAPIER, and graph construction for HYPERVISION and RETRIAL. We can see that most of the time overheads in RAPIER are attributed to label correction and data augmentation implemented by generative models. In contrast, the time-consuming operations in RETRIAL primarily involve the iterative optimization implemented by graph structure learning. The distance computations

TABLE VI
ROBUSTNESS COMPARISONS OF RETRIAL AND BASELINES WITH DIFFERENT PACKET LOSS RATES

METHODS	CONDITIONS	<i>CTU</i> (binary)		<i>MTA</i> (binary)		<i>USTC-TFC2016</i> (multi-class)	
		F1	AUC	F1	AUC	F1	AUC
JOY	$r_l = 10\%$	95.83±0.24	98.29±0.47	96.00±0.14	99.00±0.15	91.36±0.28	99.38±0.05
	$r_l = 20\%$	95.72±0.22	98.14±0.18	92.99±0.30	97.70±0.15	83.59±0.14	98.36±0.16
	$r_l = 30\%$	94.11±0.10	97.67±0.24	92.15±0.03	97.13±0.27	78.55±0.10	96.87±0.04
USTC-TK	$r_l = 10\%$	98.94±0.05	98.98±0.02	98.10±0.04	99.25±0.06	96.20±0.04	99.49±0.01
	$r_l = 20\%$	98.31±0.05	98.37±0.10	97.97±0.05	98.89±0.08	94.82±0.01	99.13±0.01
	$r_l = 30\%$	97.24±0.04	97.65±0.09	96.90±0.05	98.43±0.03	92.47±0.03	98.56±0.01
CICFLOW	$r_l = 10\%$	96.28±0.51	98.63±0.10	96.03±0.07	99.20±0.03	94.80±0.11	99.36±0.02
	$r_l = 20\%$	95.63±0.23	98.50±0.02	94.62±0.54	98.61±0.03	88.82±0.12	98.51±0.06
	$r_l = 30\%$	95.16±0.37	97.98±0.07	91.33±0.49	97.79±0.07	86.35±0.22	97.53±0.09
RS-GAT	$r_l = 10\%$	96.94±0.24	98.04±0.07	97.29±0.15	98.90±0.05	96.63±0.03	98.53±0.04
	$r_l = 20\%$	95.87±0.04	97.96±0.06	97.08±0.05	98.74±0.00	94.39±0.20	98.30±0.04
	$r_l = 30\%$	95.81±0.16	97.61±0.07	97.06±0.17	98.64±0.17	92.08±0.11	97.62±0.07
MALDISCOVERY	$r_l = 10\%$	95.50±0.09	98.61±0.12	96.98±0.09	98.93±0.01	96.32±0.07	99.41±0.05
	$r_l = 20\%$	94.48±0.09	98.51±0.08	95.12±0.14	98.35±0.09	95.18±0.24	99.37±0.06
	$r_l = 30\%$	93.02±0.07	97.97±0.04	93.34±0.09	97.39±0.10	91.87±0.07	98.53±0.08
RAPIER	$r_l = 10\%$	98.64±0.72	98.55±0.56	98.36±0.42	98.37±0.39	-	-
	$r_l = 20\%$	97.82±0.68	98.17±0.52	97.64±0.45	98.32±0.36	-	-
	$r_l = 30\%$	96.81±0.89	97.84±0.61	97.13±0.51	98.19±0.42	-	-
HYPERVISION	$r_l = 10\%$	98.15±0.00	98.62±0.00	98.47±0.00	99.38±0.00	-	-
	$r_l = 20\%$	98.03±0.00	98.41±0.00	98.17±0.00	99.13±0.00	-	-
	$r_l = 30\%$	97.92±0.00	98.21±0.00	97.81±0.00	98.85±0.00	-	-
RETRIAL	$r_l = 10\%$	99.00±0.16	99.01±0.08	98.71±0.21	99.39±0.01	97.23±0.06	99.53±0.04
	$r_l = 20\%$	98.55±0.09	98.74±0.07	98.15±0.07	99.03±0.06	97.06±0.11	99.50±0.02
	$r_l = 30\%$	97.46±0.27	98.22±0.11	97.74±0.09	98.87±0.07	96.43±0.05	98.72±0.06

TABLE VII
TIME OVERHEAD COMPARISON

Phase	RAPIER	HYPERVISION	RETRIAL
Preparation	31.89s	5.26s	4.55s
	929.77s	6.64s	228.45s
Training	8.68s	-	467.38s
Detection	0.14s	1.99s	1.17s

TABLE VIII
TIME OVERHEAD OF RETRIAL IN EACH ITERATION

Iteration	1	2	3	4	5	6	7	8	9	10
Epoch	200	89	37	58	21	34	31	22	83	21
Time (s)	132.77	80.64	31.19	47.23	16.93	28.07	25.40	17.38	68.17	19.60

during graph construction also contribute to the time overhead. Despite these differences, the robust features learned by HYPERVISION and the noise mitigation strategies for labels and interactions employed by RAPIER and RETRIAL are both critical advancements to improve detection robustness.

For further analysis, Table VIII presents the time overhead per iteration during the iterative optimization process. The epoch number is set to 200, with an early stopping criterion of 20. As the iterations progress, the GNN encoder converges earlier. On average, the time overhead per iteration is about 0.78 seconds.

C. Robustness Analysis

In this subsection, we simulate two types of adverse conditions to validate the robustness of RETRIAL and other

baselines. Notably, in adversarial attack simulation, we exclusively compare RETRIAL with graph-based baselines that take flows as nodes since the simulation is implemented by adding interactions to modify the graph.

1) *Network Fluctuations*: We randomly drop packets to simulate network fluctuations. Table VI summarizes the experimental results with varying packet loss rates.

Overall, as the packet loss rate increases, most baselines exhibit notable performance degradation due to distortion of statistical features, while RETRIAL demonstrates superior robustness in most cases. HYPERVISION also performs well, likely because packet loss primarily affects statistical features, while structural features remain largely unaffected, which play a more decisive role in ensuring detection robustness.

The performance of RAPIER declines notably under network fluctuations, as its label correction and data augmentation strategies are not designed to address the feature distortion. Similarly, JOY and CICFLOW, which rely heavily on single-flow statistical features, experience considerable performance degradation. USTC-TK, utilizing each flow's first 784 bytes, is less affected by packet loss, as each flow consists of multiple packets, and a 30% packet loss has a minor impact on its features. For RS-GAT and MALDISCOVERY, the connections between flows mitigate information loss. RETRIAL further enhances them by restoring missing relations, achieving superior performance under significant network fluctuations. In summary, the adverse effects of packet loss are less severe than expected. Both structural features and selective aggregation of contextual information are effective in mitigating packet loss.

TABLE IX
ROBUSTNESS COMPARISONS OF RETRIAL AND BASELINES WITH DIFFERENT EDGE PERTURBATION RATES

METHODS	CONDITIONS	<i>CTU</i> (binary)		<i>MTA</i> (binary)		<i>USTC-TFC2016</i> (multi-class)	
		F1	AUC	F1	AUC	F1	AUC
RS-GAT	$r_p = 5\%$	92.24±0.44	97.60±0.06	95.61±0.14	97.80±0.02	94.23±0.06	98.34±0.05
	$r_p = 10\%$	89.89±0.20	94.69±0.12	94.94±0.32	96.75±0.06	88.27±0.15	97.34±0.05
	$r_p = 15\%$	81.45±0.39	91.53±0.35	92.35±0.20	95.11±0.05	83.68±0.04	96.38±0.02
MALDISCOVERY	$r_p = 5\%$	95.44±0.09	99.07±0.05	96.13±0.19	98.95±0.09	94.69±0.20	99.18±0.10
	$r_p = 10\%$	92.94±0.17	98.44±0.21	93.94±0.21	98.35±0.06	92.71±0.10	98.86±0.04
	$r_p = 15\%$	92.96±0.21	98.29±0.07	93.46±0.34	97.84±0.10	92.12±0.10	98.29±0.04
RETRIAL	$r_p = 5\%$	98.48±0.17	99.24±0.04	98.22±0.04	99.62±0.09	96.65±0.09	99.37±0.06
	$r_p = 10\%$	97.46±0.21	98.46±0.05	97.52±0.20	99.50±0.09	94.30±0.09	99.10±0.09
	$r_p = 15\%$	96.58±0.23	98.44±0.21	95.52±0.33	98.94±0.11	93.32±0.24	98.42±0.12

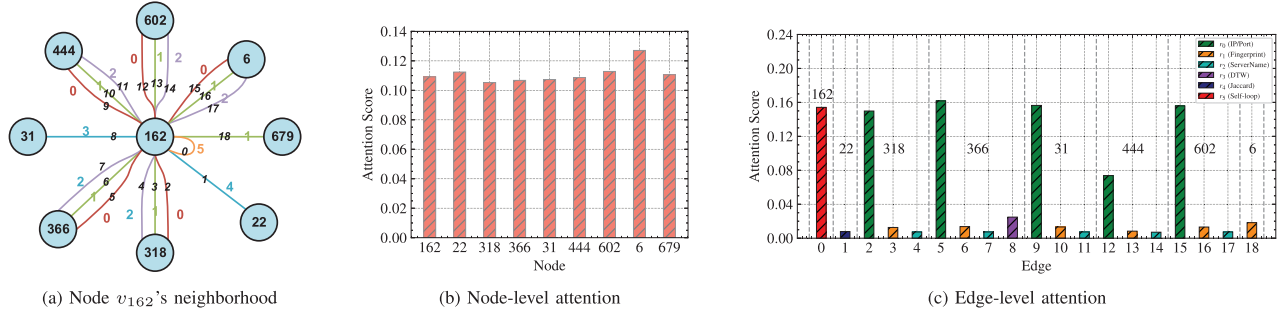


Fig. 3. Visualization of attention scores computed by the original GAT and our tailored GAT.

2) *Adversarial Attacks*: We greedily inject perturbation edges by computing the gradient of loss function to simulate adversarial attacks. Table IX summarizes the experimental results with varying edge perturbation rates.

Similarly, as the edge perturbation rate rises, the performances of RS-GAT and MALDISCOVERY experience significant degradation, whereas RETRIAL demonstrates robustness against adversarial attacks. Notably, MALDISCOVERY exhibits better robustness compared to RS-GAT. This can be attributed to the fact that MALDISCOVERY constructs a k -NN graph with a small number of edges compared to the complex graph structure in RS-GAT. Consequently, the impact on MALDISCOVERY is minimal. Though RETRIAL constructs a relational multigraph incorporating complex relations, susceptible to adversarial attacks, it maintains high detection performance in both all tasks. This underscores the efficacy of RETRIAL in correcting misleading relations, highlighting its robustness in the face of adversarial attacks.

D. Detail Analysis

RETRIAL and all baselines demonstrate satisfactory performance in both binary detection and multi-class classification scenarios, whether applied to TLS traffic or general encrypted traffic. Additionally, RETRIAL consistently exhibits stable performance under adverse conditions, in contrast to the significant degradation observed in other baselines. In this subsection, we analyze the details of RETRIAL from three aspects: attention score, graph topology, and performance.

1) *Attention Score*: RETRIAL constructs a relational multigraph with parallel edges to model complex relations in a

fine-grained manner. We design a tailored GAT that computes edge-level attention to aggregate neighbor information for contextual flow encoding. To intuitively present the improvement of RETRIAL compared to the original GAT, we take node v_{162} from *CTU* dataset with a 15% perturbation rate as an example and visualize its node-level attention scores and edge-level attention scores in Fig. 3.

As shown in Fig. 3a, node v_{162} has 8 neighbor nodes connected by 19 edges of 6 types of relations. In our relational multigraph, the existence of multiple edges between two nodes means that two flows share multiple relations. The original GAT fails to handle this situation since it merges parallel edges into one singular edge and simply computes node-level attention scores to aggregate neighbor information. In contrast, our tailored graph attention mechanism considers relation information to compute edge-level attention, achieving fine-grained neighborhood aggregation. By comparing Fig. 3b and Fig. 3c, it is apparent that node-level and edge-level attention mechanisms prioritize different neighbor nodes. Notably, node v_6 holds significant importance in node-level aggregation, while its relevance diminishes considerably in edge-level aggregation. We also observe that IP/Port, self-loop, and DTW relations are the top three relations that really matter in node v_{162} 's neighborhood aggregation. With accurate node representations, we can obtain better observations and estimate reliable graph topology for relation correction.

2) *Graph Topology*: To assess the graph topology improvement of RETRIAL, we take *CTU* dataset with a 15% edge perturbation rate as a case study, run RETRIAL with 10 iterations to estimate better graph topology. Specifically, we

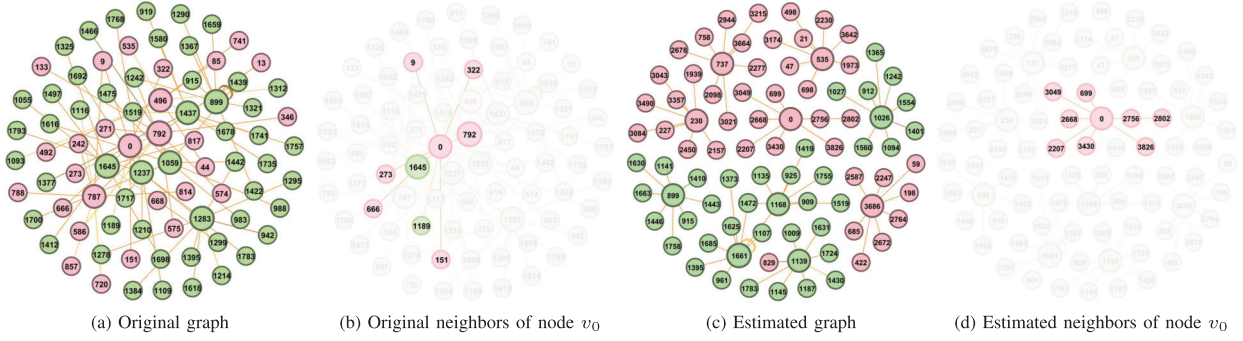


Fig. 4. Visualization of graph topology of the original multigraph and estimated multigraph after 10 iterations.

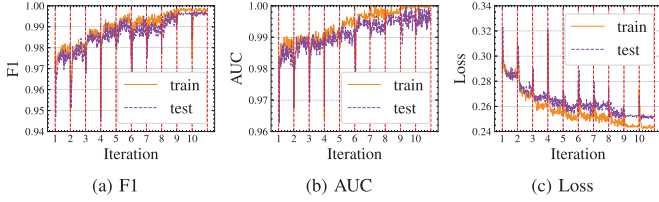


Fig. 5. Visualization of performance improvement.

sample two-order neighbors of benign node v_0 and malicious node v_{899} to visualize the original and estimated multigraph in Fig. 4, with malicious node denoted in green and benign denoted in pink.

As shown in Fig. 4a, the original multigraph displays a chaotic topology induced by a 15% edge perturbation rate simulating adversarial attacks. The perturbations introduce a multitude of connections of various relations between benign and malicious flows, which introduce noise during neighborhood aggregation, severely affecting the detection performance. Taking benign node v_0 as an example, it is observed to be connected to malicious nodes v_{1645} and v_{1189} as depicted in Fig. 4b. After 10 iterations of graph optimization as shown in Fig. 4c, we observe that many connections between benign and malicious flows are removed and the estimated graph exhibits strong homophily. Also, benign node v_0 does not have any malicious neighbors in our sampled graph in Fig. 4d. This empirical observation demonstrates that our work, RETRIAL, is proficient in deriving an improved graph topology that maintains homophily to effectively correct misleading relations and improve GNN's overall detection performance.

3) *Performance*: RETRIAL iteratively optimizes the topology of relational multigraph and the parameters of GNN encoder to achieve adaptive relation correction and robust threat detection. Here we take *MTA* dataset as a case study to see how the performance goes as the iterations increase. We run RETRIAL with 10 iterations. In each iteration, the GNN encoder is optimized with 50 epochs.

Fig. 5 illustrates the progressive performance improvement of RETRIAL with successive iterations. Generally, the detection performance of RETRIAL exhibits a consistent improvement trend. Initially, RETRIAL achieves 96.81% F1 and 98.33% AUC on the training set, along with 96.69% F1 and 98.27% AUC on the testing set. After 10 iterations of optimization, RETRIAL achieves notable improvements, reaching

99.80% F1 and 99.97% AUC on the training set, and 99.66% F1 and 99.45% AUC on the testing set. Besides, we observe that even though the performance has converged during the training of GNN encoder, further improvement is achievable after each iteration of graph optimization, which demonstrates that RETRIAL achieves robust threat detection and adaptive relation correction mutually in an iterative manner.

E. Ablation Study

We conduct a comprehensive ablation study on RETRIAL to further determine the efficacy of each key component. Specifically, we compare the full RETRIAL with three variants:

- w/o RM removes relational multigraph construction. It constructs a simple graph and does not discriminate relations. Its encoder degenerates into vanilla GAT [21].
- w/o AM removes attention mechanism, but it tackles relations by performing relation-specific transformations individually. Its encoder degenerates into R-GCN [42].
- w/o IO removes iterative optimization and solely optimize the GNN encoder by typical gradient descent.

Table X reports the results of RETRIAL and its variants on three dataset with a 30% packet loss rate ($r_l = 30\%$) and a 15% edge perturbation rate ($r_p = 15\%$).

As can be seen, RETRIAL achieves the most robust detection performance with remarkable stability across all datasets. The results of RETRIAL and w/o RM validate the efficacy of our proposed strategy for relational multigraph construction. Instead of preserving the parallel edges, w/o RM simply merge them into one singular edge, which causes notable loss of information. Furthermore, comparing RETRIAL with w/o AM, we observe that though R-GCN is proficient for modeling relational data, it fails to discriminate between different neighbors and neglects explicit consideration of relation information in neighborhood aggregation. Finally, we evaluate the iterative optimization scheme and find that RETRIAL outperforms w/o IO after several iterations of optimization of both the GNN encoder and the graph topology. Overall, RETRIAL outperforms all variants by integrating the rich relations, fine-grained attention mechanism, and iterative optimization based on Bayesian inference and EM optimization, which substantiates the contributions of three key components for the robustness of RETRIAL.

Moreover, we observe that the performance of RETRIAL variants significantly varies across binary detection and

TABLE X
ABLATION STUDY OF RETRIAL AND ITS VARIANTS ON USTC-TFC2016 DATASET

VARIANTS	CONDITIONS	<i>CTU</i> (binary)		<i>MTA</i> (binary)		<i>USTC-TFC2016</i> (multi-class)	
		F1	AUC	F1	AUC	F1	AUC
w/o RM	$r_l = 30\%$	95.95±0.42	97.97±0.36	95.81±0.09	96.83±0.05	91.33±0.17	97.73±0.03
	$r_p = 15\%$	93.87±0.28	96.94±0.30	94.41±0.21	94.45±0.15	77.19±0.71	96.51±0.12
w/o AM	$r_l = 30\%$	96.44±0.19	97.17±0.16	96.64±0.13	97.57±0.13	88.11±0.07	93.81±0.04
	$r_p = 15\%$	95.43±0.26	97.70±0.23	94.94±0.26	96.48±0.11	77.17±0.42	92.84±0.30
w/o IO	$r_l = 30\%$	93.61±0.28	97.21±0.29	94.71±0.04	95.90±0.05	93.35±0.18	98.06±0.14
	$r_p = 15\%$	92.15±0.17	96.62±0.26	92.46±0.13	96.22±0.24	91.12±0.19	98.20±0.24
RETRIAL	$r_l = 30\%$	97.46±0.27	98.22±0.11	97.74±0.09	98.87±0.07	96.43±0.05	98.72±0.06
	$r_p = 15\%$	96.58±0.23	98.44±0.21	95.52±0.33	98.94±0.11	93.32±0.24	98.42±0.12

multi-class classification tasks. On *CTU* and *MTA* datasets (binary detection), iterative optimization (IO) is the most critical component to improve the robustness compared to relation multigraph (RM) and attention mechanism (AM). However, RM and AM contribute more on *USTC-TFC2016* dataset (multi-class classification) than IO. These results indicate that iterative optimization is more effective in tasks with fewer classes. Conversely, in a more complex classification tasks with a larger number of communities, the efficacy of iterative optimization is heavily dependent on the accuracy of the graph topology and the performance of GNN encoder.

In summary, the combination of IO, RM and AM makes RETRIAL robust to defend against network fluctuations and adversarial attacks. IO contributes more in binary detection tasks, while RM and AM are more effective in multi-class classification tasks.

VI. DISCUSSION

A. Homophily Assumption

Retrial employs a tailored GAT as the backbone GNN, where both the GNN encoder and the graph estimator are built upon an ideal homophily assumption, that is, nodes with similar embeddings and identical labels tend to connect. However, this assumption does not always hold in real-world graphs, and the performance of GNN models on such graphs may degrade significantly and might be even worse than a Multi-Layer Perceptron (MLP) model that does not leverage any contextual information. In this work, we modify the relational multigraph to maintain good homophily and stimulate the GNN encoder's potential. In the face of severe security threats and rapidly evolving evasion techniques, we consider to integrate advanced GNNs tailored for heterophilous graphs for better generalization.

B. Efficiency Concerns

RETRIAL constructs a relational multigraph to model the rich relations among flows and optimizes the graph topology to correct those spurious ones, which prioritizes retroactive analysis over real-time alerts. The efficiency of RETRIAL may encounter limitations in large-scale networks. As the number of flows increases, the graph construction and inference time of RETRIAL significantly escalate. Future efforts could explore flow aggregation algorithms to reduce graph scale, and integrate high-performance implementations of graph construction and distance calculation to improve the efficiency.

C. Robustness Analysis

RETRIAL, RAPIER, and HYPERVISION are all designed for robust detection with different emphases. RAPIER aims to reach the full potential of low-quality training data by correcting noisy labels and synthesizing new training data during preparation phase. HYPERVISION utilizes graph connectivity, sparsity, and statistical features to model robust flow interaction patterns. In RETRIAL, we focus on accurately modeling the rich relations among flows and adaptively correct those induced by evasion techniques during training and testing phase. A promising future direction is to integrate label correction and robust features along with relation correction to further improve the robustness across all phases.

D. Topology Considerations

RETRIAL aggregates neighbor information in contextual flow encoding component to implicitly incorporate topological information. However, HYPERVISION has demonstrated that some ignored structural features (e.g., node in-degrees and out-degrees) are effective for malicious traffic detection. In future work, we could explicitly encode node degrees as node features for neighborhood aggregation. Moreover, exploring alternative graph construction strategies, such as communication graphs that naturally model the interactions between hosts, could also improve detection efficiency and efficacy.

VII. CONCLUSION

In this paper, we present RETRIAL, a robust encrypted malicious traffic detection system that can effectively tackle adversarial attacks and evasion techniques. The key motivations behind RETRIAL are to adaptively integrate and correct various interactions, mitigating adverse effects induced by evasion techniques and generating robust flow features for robust detection. Specifically, RETRIAL constructs a relational multigraph and adopts a tailored GAT to selectively aggregate neighbor information. By taking intermediate node embeddings to construct k -NN graphs as observations, RETRIAL estimates the optimal graph based on Bayesian inference. Following an iterative optimization scheme, RETRIAL alternatively optimizes the GNN encoder by typical gradient descent for robust threat detection and refines the relational graph by EM algorithm for adaptive relation correction. Extensive experiments demonstrate the superior performance of

RETRIAL in encrypted malicious traffic detection, even in the face of network fluctuations and adversarial attacks.

REFERENCES

- [1] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.
- [2] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [3] D. Han et al., "DeepAID: Interpreting and improving deep learning-based anomaly detection in security applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3197–3217.
- [4] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Def.*, 2022, pp. 495–509.
- [5] C. Qiu et al., "3D-IDS: Doubly disentangled dynamic intrusion detection," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2023, pp. 1965–1977.
- [6] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.
- [7] J. A. Lindeman and Laura, "TLS fingerprinting with JA3 and JA3S," Salesforce, Inc., San Francisco, CA, USA, Tech. Rep., Jan. 2019. [Online]. Available: <https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/>
- [8] J. A. Lindeman and Laura, "Easily identify malicious servers on the Internet with JARM," Salesforce, Inc., San Francisco, CA, USA, Tech. Rep., Nov. 2020. [Online]. Available: <https://engineering.salesforce.com/easily-identify-malicious-servers-on-the-internet-with-jarm-e095edac525a/>
- [9] B. Anderson and D. McGrew, "Identifying encrypted malware traffic with contextual flow data," in *Proc. ACM Workshop Artif. Intell. Secur.*, Vienna, Austria, Oct. 2016, pp. 35–46.
- [10] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 1723–1732.
- [11] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Comput. Virol. Hacking Techn.*, vol. 14, no. 3, pp. 195–211, Aug. 2018.
- [12] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [13] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
- [14] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.
- [15] Y. Qing et al., "Low-quality training data only? A robust framework for detecting encrypted malicious network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–18.
- [16] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA: Internet Society, 2023, pp. 1–18.
- [17] D. Han et al., "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2632–2647, Aug. 2021.
- [18] M. Wei, "Domain shadowing: Leveraging content delivery networks for robust blocking-resistant communications," in *Proc. 30th USENIX Security Symp. (USENIX Security)*, 2021, pp. 3327–3343.
- [19] A. M. Sadeghzadeh, S. Shiravi, and R. Jalili, "Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1962–1976, Jun. 2021.
- [20] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, "TANTRA: Timing-based adversarial network traffic reshaping attack," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3225–3237, 2022.
- [21] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [22] Z. Durumeric et al., "The security impact of HTTPS interception," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA: Internet Society, 2017, pp. 1–14.
- [23] F. Wilkens, S. Haas, J. Amann, and M. Fischer, "Passive, transparent, and selective TLS decryption for network security monitoring," 2021, *arXiv:2104.09828*.
- [24] S. Ramanathan, J. Mirkovic, and M. Yu, "BLAG: Improving the accuracy of blacklists," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA: Internet Society, 2020, pp. 1–15.
- [25] C. Dong, Z. Lu, Z. Cui, B. Liu, and K. Chen, "MBTree: Detecting encryption RATs communication using malicious behavior tree," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3589–3603, 2021.
- [26] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [27] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [28] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [29] M. Trevisan, F. Soro, M. Mellia, I. Drago, and R. Morla, "Does domain name encryption increase users' privacy?," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 50, no. 3, pp. 16–22, Jul. 2020.
- [30] R. Dai, C. Gao, B. Lang, L. Yang, H. Liu, and S. Chen, "SSL malicious traffic detection based on multi-view features," in *Proc. 9th Int. Conf. Commun. Netw. Secur.*, Chongqing, China, Nov. 2019, pp. 40–46.
- [31] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.
- [32] J. Zhao, Q. Li, S. Liu, Y. Yang, and Y. Hong, "Towards traffic supervision in 6G: A graph neural network-based encrypted malicious traffic detection method," *SCIENTIA SINICA Informationis*, vol. 52, no. 2, pp. 270–286, Feb. 2022.
- [33] Y. Hong, Q. Li, Y. Yang, and M. Shen, "Graph based encrypted malicious traffic detection with hybrid analysis of multi-view features," *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119229.
- [34] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.
- [35] H. Yan et al., "Automatic evasion of machine learning-based network intrusion detection systems," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 153–167, Jan. 2024.
- [36] M. Shen et al., "Real-time website fingerprinting defense via traffic cluster anonymization," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 3238–3256.
- [37] R. Wang et al., "Graph structure estimation neural networks," in *Proc. Web Conf.*, Apr. 2021, pp. 342–353.
- [38] Q. Lv et al., "Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks," 2021, *arXiv:2112.14936*.
- [39] E. Abbe, "Community detection and stochastic block models: Recent developments," *J. Mach. Learn. Res.*, vol. 18, no. 177, pp. 1–86, 2018.
- [40] R. Liao et al., "Efficient graph generation with graph recurrent attention networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 4255–4265.
- [41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy*, Porto, Portugal: SciTePress, 2017.
- [42] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.*, 2018, pp. 593–607.